



UT1. MANEJO DE FICHEROS

ACTIVIDAD 7

Marshalling de objeto Java a archivo XML

Título: Serializa el objeto Java “Usuario” a un archivo XML con la API JAXB.

Descripción:

En esta actividad vamos a **serializar (marshalling)** un objeto Java que representa a un **usuario de un sistema de seguridad** en un fichero XML.

El objetivo es aprender a convertir un objeto Java en un documento XML estándar utilizando la API **JAXB**, de forma que los datos de un usuario (id, nombre, email y rol) puedan almacenarse o compartirse en un formato estructurado. Esta técnica es habitual en aplicaciones de **ciberseguridad y gestión de usuarios**, donde es necesario exportar información de manera portable y clara.

Objetivos:

- Comprender el proceso de marshalling para convertir un objeto Java en XML.
- Usar las anotaciones de JAXB para definir cómo se genera el XML.
- Practicar la escritura de datos en un fichero XML a partir de un objeto Java.

Instrucciones:

1. Preparación del entorno:

- Crea un nuevo proyecto Java en IntelliJ con el nombre **MarshalUsuario_JAXB**.

2. Define la clase Usuario:

- Crea una clase Java llamada **Usuario**.
- Define los siguientes atributos en la clase **Usuario**:
 - **int id**
 - **String nombre**
 - **String email**
 - **String rol**
- Proporciona un constructor vacío (obligatorio para JAXB), un constructor con parámetros, y getters y setters para cada uno de estos atributos.
- Anota el atributo id con **@XmlAttribute** para que se serialice como atributo XML, y el resto de atributos (nombre, email y rol) con **@XmlElement** para que se serialicen como elementos XML.

3. Clase principal para el marshalling / Serialización a XML:

- Crea una clase llamada **SerializarUsuario** con un método main().
- En el main(), crea un objeto Usuario con valores de ejemplo.
- Usa JAXBContext y Marshaller para convertir el objeto a XML.

- Guarda el resultado en un archivo llamado usuario.xml en el directorio del proyecto.

4. Verificación:

- Revisa el archivo usuario.xml para comprobar que el XML generado refleja correctamente los datos del usuario..

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Se presenta la tarea y el código es correcto, pero no se demuestra su ejecución con evidencias (capturas o logs de consola).
7	Se presenta la tarea, el código es correcto, y se demuestra el funcionamiento mediante capturas o salida de consola; sin embargo, no se describen los principales métodos y clases del programa.
9	Se completa la tarea correctamente en todos sus apartados, con código funcional, capturas de consola y breve descripción de métodos y clases.
10	Se añade alguna funcionalidad extra o detalle innovador que mejore la aplicación (por ejemplo, añadir fecha/hora de creación del XML o escribir varios usuarios).

Entrega:

- Incluye aquí el código fuente de las clases desarrolladas (Usuario.java y SerializarUsuario.java):

```
package org.example;

import jakarta.xml.bind.annotation.*;

// Anotaciones de la clase
@XmlRootElement(name = "usuario")
@XmlAccessorType(XmlAccessType.FIELD)

public class Usuario {
    @XmlAttribute
    int id;
    String nombre;
    String email;
    String rol;

    public Usuario() {
    }

    public Usuario(int id, String nombre, String email, String rol) {
        this.id = id;
        this.nombre = nombre;
        this.email = email;
        this.rol = rol;
    }
}
```



```
}

// Getters
public int getId() {
    return id;
}
public String getNombre() {
    return nombre;
}
public String getEmail() {
    return email;
}
public String getRol() {
    return rol;
}

// Setters
public void setId(int id) {
    this.id = id;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
public void setEmail(String email) {
    this.email = email;
}
public void setRol(String rol) {
    this.rol = rol;
}
}
```

```
package org.example;

import jakarta.xml.bind.*;
import java.io.File;

public class SerializarUsuario {
    private static final String FICHERO = "usuario.xml";

    public static void main (String []args) {
        Usuario usuario = new Usuario(1,"Loren", "loren@gmail.com","administrador");

        try {
            // 2. Crear el contexto de JAXB asociándolo a la clase Usuario
            JAXBContext context = JAXBContext.newInstance(Usuario.class);

            // 3. Crear el Marshaller
            Marshaller marshaller = context.createMarshaller();

            // Configurar para que el XML tenga sangría / formato legible
            marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, Boolean.TRUE);

            // 4. Definir el archivo de salida 'usuario.xml'
```

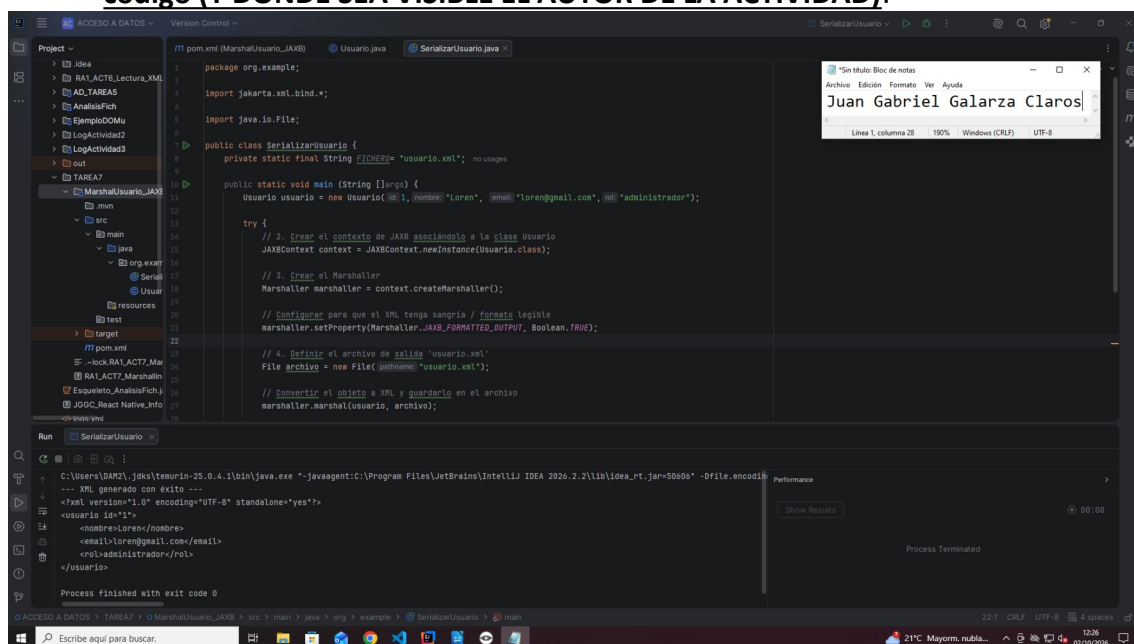
```
File archivo = new File("usuario.xml");

// Convertir el objeto a XML y guardarlo en el archivo
marshaller.marshal(usuario, archivo);

// También lo mostramos por consola para confirmar
System.out.println("--- XML generado con éxito ---");
marshaller.marshal(usuario, System.out);

} catch (JAXBException e) {
    System.err.println("Error durante la serialización a XML: " + e.getMessage());
    e.printStackTrace();
}
}
```

- Incluye aquí captura/s de pantalla que demuestren la correcta ejecución del código (Y DONDE SEA VISIBLE EL AUTOR DE LA ACTIVIDAD):



- Describe aquí las clases, métodos y variables principales utilizadas en el programa:

Clase

- **Usuario:** Clase principal del modelo. Utiliza la anotación `@XmlRootElement(name = "usuario")` para definir que esta clase corresponde al nodo raíz `<usuario>` en un documento XML, y `@XmlAccessorType(XmlAccessType.FIELD)` para indicarle a JAXB que lea las anotaciones directamente sobre sus campos/atributos.



Variables (Atributos)

- **id** (int): Identificador único del usuario. Contiene la anotación `@XmlAttribute`, por lo que en el XML se serializa como un atributo de la etiqueta raíz (ejemplo: `<usuario id="1">`).
- **nombre** (String): Nombre completo o de cuenta del usuario. Se mapea como un elemento hijo `<nombre>`.
- **email** (String): Dirección de correo electrónico. Se mapea como un elemento hijo `<email>`.
- **rol** (String): Rol o tipo de permisos que tiene el usuario dentro del sistema (ej. "Admin", "User"). Se mapea como un elemento hijo `<rol>`.

Métodos Principales

Constructores

- **Usuario()**: Constructor por defecto (sin parámetros). Es **obligatorio** para que el motor de JAXB pueda instanciar la clase al deserializar un XML a un objeto Java.
- **Usuario(int id, String nombre, String email, String rol)**: Constructor parametrizado para facilitar la creación de instancias con todos sus atributos inicializados desde el código.

Métodos Getter y Setter

- **Getters (`getId()`, `getNombre()`, `getEmail()`, `getRol()`)**: Métodos de acceso para obtener los valores de las propiedades privadas/protegidas de la clase.
- **Setters (`setId()`, `setNombre()`, `setEmail()`, `setRol()`)**: Métodos de modificación para establecer o actualizar los valores de las propiedades.

Clase

- **SerializarUsuario**: Clase principal del proceso de serialización. Contiene la lógica para configurar el contexto de JAXB, transformar el objeto Usuario en etiquetas XML y escribir el resultado tanto en el disco como en la consola.

Variables (Atributos)

- **FICHERO** (private static final String): Constante que almacena la ruta y el nombre del archivo de salida ("usuario.xml").

Métodos Principales

- **main(String[] args)**: Método punto de entrada del programa. Ejecuta la lógica completa de serialización:
 1. Instancia un objeto Usuario con datos de prueba.
 2. Inicializa el JAXBContext asociado a la clase Usuario.
 3. Crea y configura un objeto Marshaller (activando `JAXB_FORMATTED_OUTPUT` para que el XML resultante tenga sangría y sea fácil de leer).

4. Llama al método `marshal()` para escribir el objeto en el archivo `usuario.xml` y mostrar el resultado por pantalla.
5. Captura posibles excepciones de tipo `JAXBException` en caso de error durante el proceso.